**VIETNAM NATIONAL UNIVERSITY — HO CHI MINH CITY**
**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY**
**FACULTY OF COMPUTER SCIENCE AND ENGINEERING**


## INTRODUCTION TO ARTIFICIAL INTELLIGENCE

---

### Assignment 2 Report

# Search-Based Xiangqi Agent (Depth Search and Heuristic Search)

---

**Instructor:** Dr. Tran Hong Tai

**Group:** NewBoyAI

**Members:**

Duong Dang Khoa — [2352551]
Tran Hoang Vy Khang — [2352502]
Nguyen Duc Gia Bao — [2352095]
Dong Cong Khanh — [2352517]
Nguyen Duc Hanh Nhi — [2252578]

Ho Chi Minh City, April 2026

# Contents

# 1 Objectives

This assignment focuses on building a **search-based game-playing agent** for **Xiangqi (Chinese Chess)**. The implementation targets a large state-space adversarial game and satisfies the project requirements without using machine learning.

Main objectives:

- Implement a rule-correct Xiangqi engine (state, moves, legality, terminal conditions).
- Implement and compare two search paradigms:
  - **Depth Search:** Minimax
  - **Heuristic Search:** Alpha-Beta with evaluation function and move ordering
- Provide multiple AI levels (Easy, Medium, Hard).
- Support practical evaluation through automated tests and benchmarking scripts.

# 2 Introduction

Xiangqi is a deterministic, perfect-information, turn-based zero-sum game with a high branching factor and deep tactical interactions. Exhaustive search is infeasible in practice, so the project applies selective game-tree search with domain heuristics.

The system is implemented in Python with a modular architecture:

- **core/**: rules, board/state representation, legal move generation.
- **agents/**: random agent, minimax/alpha-beta search agents, level wrappers.
- **game/**: backend game loop and match execution.
- **ui/**: Pygame visualization and interaction.
- **tests/**: unit tests for correctness and regression safety.
- **evaluation/**: script-based evaluation and result export.

# 3 Game Definition and Problem Formulation

## 3.1 Adversarial Search Model

Xiangqi is modeled as a two-player deterministic zero-sum game:

$$\langle S, A, T, U \rangle$$

where:

- $S$ is the set of valid game states,
- $A(s)$ is the legal action set at state $s$,
- $T(s,a)$ is the transition function,
- $U(s)$ is utility at terminal states (win/loss/draw).

The search agents do not learn from data; they approximate best action by exploring the game tree from current state and applying a handcrafted evaluation at cut-off depth.

## 3.2 Rules and Terminal Conditions

The implementation enforces Xiangqi movement rules for all piece types, including constraints such as palace boundaries, river crossing behavior, horse leg blocking, cannon screen captures, and facing-generals restriction.

Terminal handling includes:

- General captured,
- Checkmate,
- Stalemate,
- Threefold repetition draw (to avoid infinite loops).

## 3.3 State Representation

A game state includes:

- Board piece layout,
- Side to move,
- Move history.

For each search expansion, successor states are generated from legal moves only. This keeps the tree valid by construction and simplifies terminal checks.

## 3.4 Core Engine Design Decisions

To keep the engine robust for both gameplay and search, the implementation follows these decisions:

- **Immutable move objects**: move records are frozen to avoid accidental mutation.
- **Explicit apply/undo path**: search recursion can safely traverse and roll back states.
- **Single legality source**: all agents rely on the same `legal_moves` API.
- **Terminal-first handling**: checkmate/general-capture/stalemate/repetition are checked consistently in loop and search utility.

This architecture prevents divergence between UI behavior and backend search behavior.

# 4 Search Algorithms

## 4.1 Depth Search: Minimax

Minimax searches the game tree by alternating maximizing and minimizing layers:

$$V(s) = \begin{cases} \max\limits_{a \in A(s)} V(T(s,a)), & \text{if max node} \\ \min\limits_{a \in A(s)} V(T(s,a)), & \text{if min node} \end{cases}$$

In this project, Minimax is used as the depth-search baseline and appears explicitly in AI-vs-AI mode for comparison.

**How Minimax is implemented in this project** The implementation follows this sequence:
1. Generate all legal moves from current state (using `legal_moves` (state)).
2. Apply one move to create a child state.
3. Recursively evaluate child states until depth limit.
4. At leaf level:
   - If terminal position: use terminal utility with very large magnitude.
   - Otherwise: use evaluation function score.
5. Back-propagate by `max` at maximizing nodes and `min` at minimizing nodes.

**Complexity perspective** If branching factor is $b$ and search depth is $d$, plain Minimax complexity is approximately:

$$O(b^d)$$

In Xiangqi, this grows quickly; therefore Minimax is mainly used as a depth-search reference and comparison baseline.

**Strengths and limitations**

- Strength: conceptually simple, deterministic baseline for depth search.
- Limitation: explores many unnecessary nodes without pruning, so slower at deeper levels.

## 4.2 Heuristic Search: Alpha-Beta

Alpha-Beta pruning reduces the number of expanded nodes while preserving Minimax-optimal decisions:

- $\alpha$: best lower bound for maximizing player,
- $\beta$: best upper bound for minimizing player,
- prune when $\alpha \geq \beta$.

Move ordering is used to improve pruning efficiency, especially at medium/hard levels.

**How Alpha-Beta is implemented in this project** The engine extends Minimax with three practical optimizations:

- **Pruning**: skip subtrees when bounds prove they cannot affect final decision.
- **Move ordering**: prioritize tactical moves first (capture priority), improving early cut-offs.
- **Caching**: transposition/evaluation caches reduce repeated computation for equivalent states.

**Practical effect** Compared with pure Minimax at the same depth, Alpha-Beta typically searches fewer nodes and returns a move faster. This is why Alpha-Beta is used as the primary heuristic-search agent.

**Complexity perspective** Worst-case complexity remains $O(b^d)$, but with effective move ordering the practical explored-node count is significantly reduced. In favorable ordering cases, the effective search depth achievable under fixed time budget is much higher than plain Minimax.

### 4.3 Evaluation Function

The heuristic evaluation combines:

- Material values (general, rook, cannon, horse, advisor, elephant, soldier),
- Positional adjustments (for example, soldier advancement and piece activity).

Terminal utility is assigned a very large magnitude to prioritize forced wins and avoid losing lines:

$$U_{terminal} = \pm M, \quad M \gg \text{material scale}$$

**Why terminal utility is critical** Without strong terminal utility, an agent may prefer local material gain over forced checkmate lines. Assigning very large terminal utility ensures tactical correctness: immediate forced win is always prioritized over non-terminal heuristic improvements.

### 4.4 Algorithm Assignment in Modes

- **Human vs AI**: AI side uses search agent with selected level.
- **AI vs Random**: search agent vs random baseline for capability comparison.
- **AI vs AI**: Red uses Alpha-Beta (heuristic search), Black uses Minimax (depth search), with separate side-level selection.

This mapping is intentionally chosen to satisfy Assignment 2's requirement of demonstrating both depth search and heuristic search in gameplay.

## 5 Agent Modes and Difficulty Design

### 5.1 Game Modes

- **Human vs AI**: human controls Red, AI controls Black.
- **AI vs Random**: search AI (Red) versus random baseline (Black).
- **AI vs AI**: two search AIs with separate side configuration.

### 5.2 Assignment 2 Search Configuration

In AI-vs-AI demonstration mode:

- **Red AI**: Heuristic Search (Alpha-Beta)
- **Black AI**: Depth Search (Minimax)

The UI supports separate Red/Black level selection in this mode.

### 5.3 Difficulty Levels

Levels are mapped to search depth and search features:

- Easy: shallow search.
- Medium: deeper search with stronger evaluation.
- Hard: strongest practical depth with ordering.

This provides a clear poor-average-good progression as required by the assignment.

### 5.4 Assignment Requirement Traceability

The implementation-to-requirement mapping is summarized below:

- **Play by rules**: enforced by `core/rules.py`, `core/move_generator.py`, and rule-focused tests.
- **Defeat random baseline**: supported by AI-vs-Random mode and evaluation script automation.
- **Multiple skill levels**: Easy/Medium/Hard with level-dependent search profile.
- **Search only**: Minimax and Alpha-Beta only, no ML model/training pipeline.
- **Evaluation workflow**: automated tests + evaluation export (JSON/CSV).

## 6 Validation and Evaluation

### 6.1 Automated Tests

The project includes test suites for rules, moves, state transitions, game loop behavior, and search agents.

#### 6.1.1 What is tested

Table 1: Automated test coverage by module

| Test file | Focus | Examples |
|---|---|---|
| `tests/test_moves.py` | Move object correctness | capture flag, unpacking, immutability |
| `tests/test_rules.py` | Legal rules and terminal logic | horse-leg block, cannon screen, check/checkmate |
| `tests/test_state.py` | GameState consistency | apply/undo, clone independence, move history |
| `tests/test_game_loop.py` | Loop orchestration | turn order, illegal move rejection, mode flow |
| `tests/test_agents.py` | Search-agent behavior | legal move output for Easy/Medium/Hard/Minimax |

#### 6.1.2 Step-by-step test procedure

**Step 1: Syntax validation**

```
python -m py_compile main.py ui/game_ui.py agents/search_agent.py game/
   game_loop.py
```

Listing 1: Syntax check before running tests

Expected result: no output (success).

**Step 2: Run full regression test suite**

```
python -m unittest discover -s tests -p "test_*.py"
```

Listing 2: Run all tests

Expected result:

```
Ran 43 tests in <time>
OK
```

**Step 3: Run focused search-agent tests**

```
python -m unittest tests.test_agents
```

Listing 3: Run search-agent tests only

Expected result: all agent tests pass and returned moves are legal.

**Step 4: Manual gameplay smoke test**

```
python main.py
```

Listing 4: Start game UI

Checklist:

- Switch among modes (Human vs AI, AI vs AI, AI vs Random).
- Change levels and verify turn progression.
- Confirm endgame banner appears (win/draw).
- Confirm threefold repetition terminates looped positions.

**Step 5: Evaluation pipeline**

```
python evaluation/evaluate.py
```

Listing 5: Run evaluation and export results

Expected result: summary printed in terminal, and JSON/CSV files generated in `evaluation/results/`.

### 6.1.3 Benchmark protocol (reproducible)

For report/demo consistency, benchmarking should use a fixed protocol:

1. Run paired-color matches (swap Red/Black roles) for fairness.
2. Keep fixed max-turn threshold to avoid infinite runs.
3. Record: winner, reason, plies, elapsed time.
4. Export one JSON and one CSV artifact per run.

This protocol is implemented in `evaluation/evaluate.py`.

### 6.1.4 Current observed automated-test result

At the current project state, the full suite executes successfully:

```
.........................................
----------------------------------------------------------------------
Ran 43 tests in 115.334s

OK
```

## 6.2 Evaluation Script

The evaluation pipeline supports automated match batches and exports results to JSON/CSV.

Example command:

```
python evaluation/evaluate.py
```

Listing 6: Run evaluation script

Outputs are written to `evaluation/results/` for later analysis and demo reporting.

## 6.3 Observed Quantitative Signals

From the current implementation behavior:

- Full regression testing is stable (43 tests pass).
- Search-agent tests confirm all configured levels and both algorithms return legal moves.
- Evaluation runs generate valid structured artifacts (JSON/CSV), enabling external analysis and charting.

Although many symmetric AI-vs-AI positions tend to end in draws (especially with repetition protection), this is expected in deterministic adversarial settings when both sides are similarly strong and tactical horizon is limited.

## 6.4 Detailed Test Strategy

To ensure reliability, testing is split into four layers:
1. **Unit rule tests**: verify local chess rules and movement constraints.
2. **State-transition tests**: verify apply/undo and cloning consistency.
3. **Integration loop tests**: verify multi-agent interaction and turn execution.
4. **Scenario tests**: verify mode-specific behavior and terminal outcomes.

**Why this strategy is sufficient** Xiangqi errors usually come from either (i) illegal move generation, (ii) invalid turn transitions, or (iii) search selecting illegal states. The selected test layers directly target these failure classes.

## 6.5 Result Interpretation

- Passing `test_rules.py` indicates core movement and check logic are stable.
- Passing `test_state.py` indicates state rollback and history are reliable for search.
- Passing `test_agents.py` indicates all AI levels/algorithms produce legal actions.
- Passing `test_game_loop.py` indicates mode orchestration and game progression are correct.

Therefore, a full green suite gives strong confidence that gameplay, search, and orchestration are correct for Assignment 2 scope.

## 6.6 Failure Modes and Mitigations

Common practical issues and implemented mitigations:

- **Looping behavior**: mitigated by threefold repetition draw handling.
- **Slow deep search**: mitigated by Alpha-Beta pruning and move ordering.
- **UI/backend mismatch**: mitigated by routing all move legality through backend validators.
- **False tactical preference**: mitigated by large terminal utility scoring.

## 6.7 Requirement Alignment

The implementation addresses assignment constraints:

- Rule-correct Xiangqi gameplay.
- Search-only AI methods (no machine learning).
- Multiple AI levels and practical comparison modes.
- Repeatable testing and evaluation scripts.

# 7 Conclusion

This Assignment 2 implementation demonstrates a complete search-based Xiangqi AI system with both depth search (Minimax) and heuristic search (Alpha-Beta). The project combines correct game modeling, practical agent design, and reproducible validation.

Key outcomes:

- A working adversarial Xiangqi engine with legal move control and terminal detection.
- Distinct search strategies for comparative analysis.
- Difficulty tiers suitable for demonstration and benchmarking.
- A maintainable codebase with automated tests and evaluation outputs.

**Future Work** Potential improvements beyond assignment scope:

- Iterative deepening with time control per move.
- Better tactical move ordering (checks, captures, threats).
- Opening book and endgame table heuristics.
- Richer benchmark dashboard from exported JSON/CSV results.